Interfaces

interface

method declaration

int sum(int a, int b);

- **interface** fields are public, static and final by default, and methods are public and abstract.

Abstract Method

Inside class

```
interface Printable{
int MIN=5;
void print();
}
```

Printable.java


```
interface Printable{
public static final int MIN=5;
public abstract void print();
}
```

Printable.class

abstract

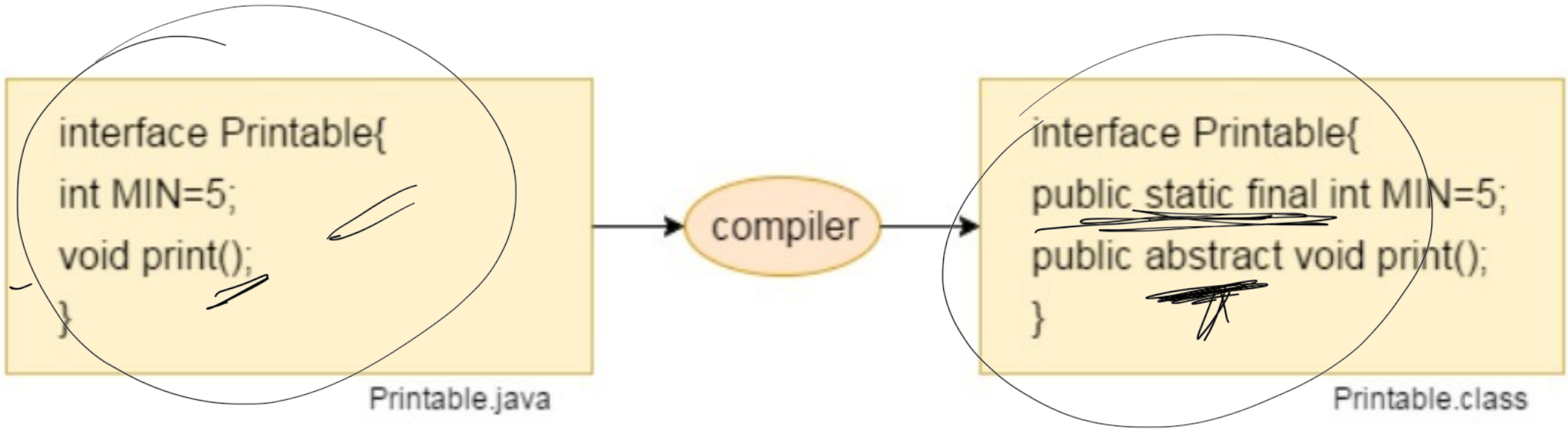
Can't be instantiated

Properties of **interfaces**

- The **interface** keyword is used to declare an interface.
- Interfaces have the following properties:
  - An interface is implicitly abstract. We do not need to use the **abstract** keyword when declaring an interface.
  - Each method in an interface is also implicitly abstract, so the abstract keyword is not needed.
  - **Methods in an interface are implicitly public.**
  - **Variable in an interface are implicitly public, static & final** (and have to be assigned values there).

interface

- **interface** fields are public, static and final by default, and methods are public and abstract.



Implementing interfaces

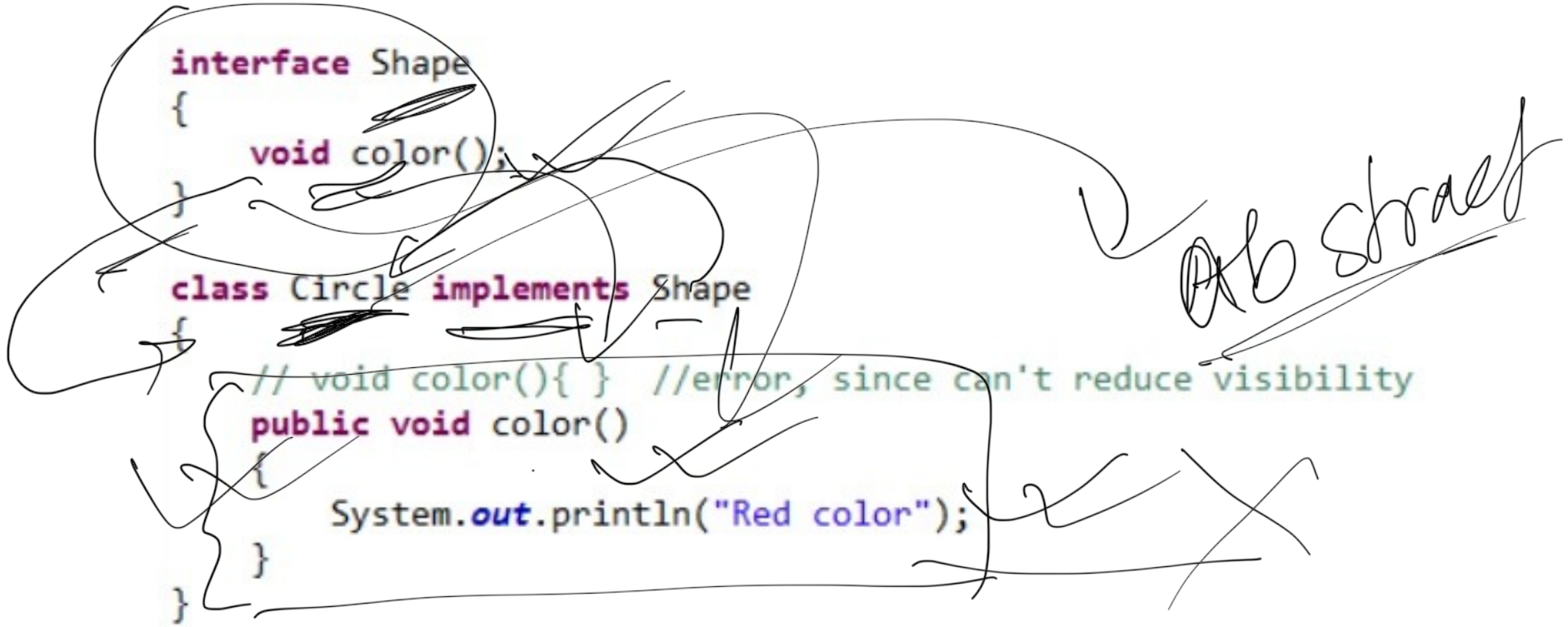
- When a class implements an interface, then it has to perform the specific behaviors of the interface.
- If a class does not perform all the behaviors of the interface, the class must declare itself as abstract.
- A class uses the **implements** keyword to implement an interface.
- The implements keyword appears in the class declaration following the extends portion of the declaration.

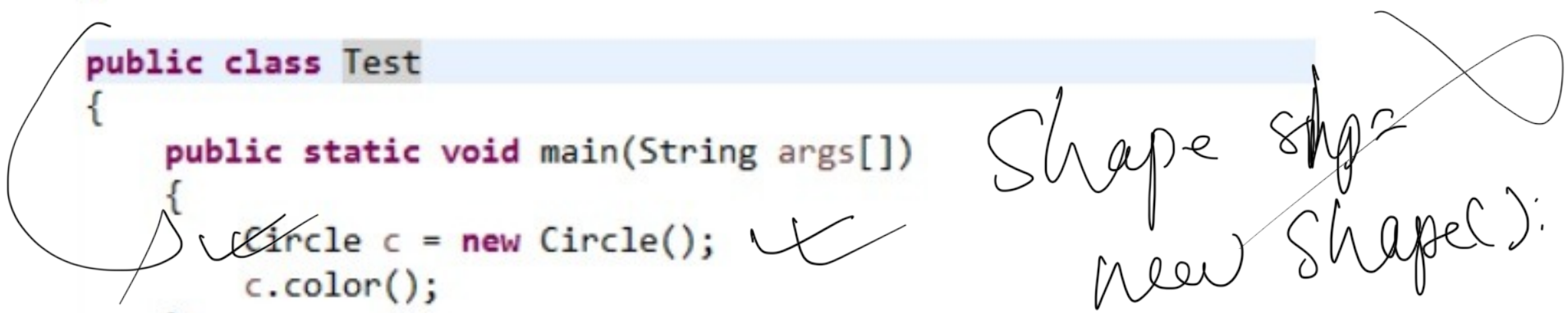
Example (implementing interface)

```
interface Shape
{
    void color();
}

class Circle implements Shape
{
    // void color(){ }  //error, since can't reduce visibility
    public void color()
    {
        System.out.println("Red color");
    }
}

public class Test
{
    public static void main(String args[])
    {
        Circle c = new Circle();
        c.color();
    }
}
```



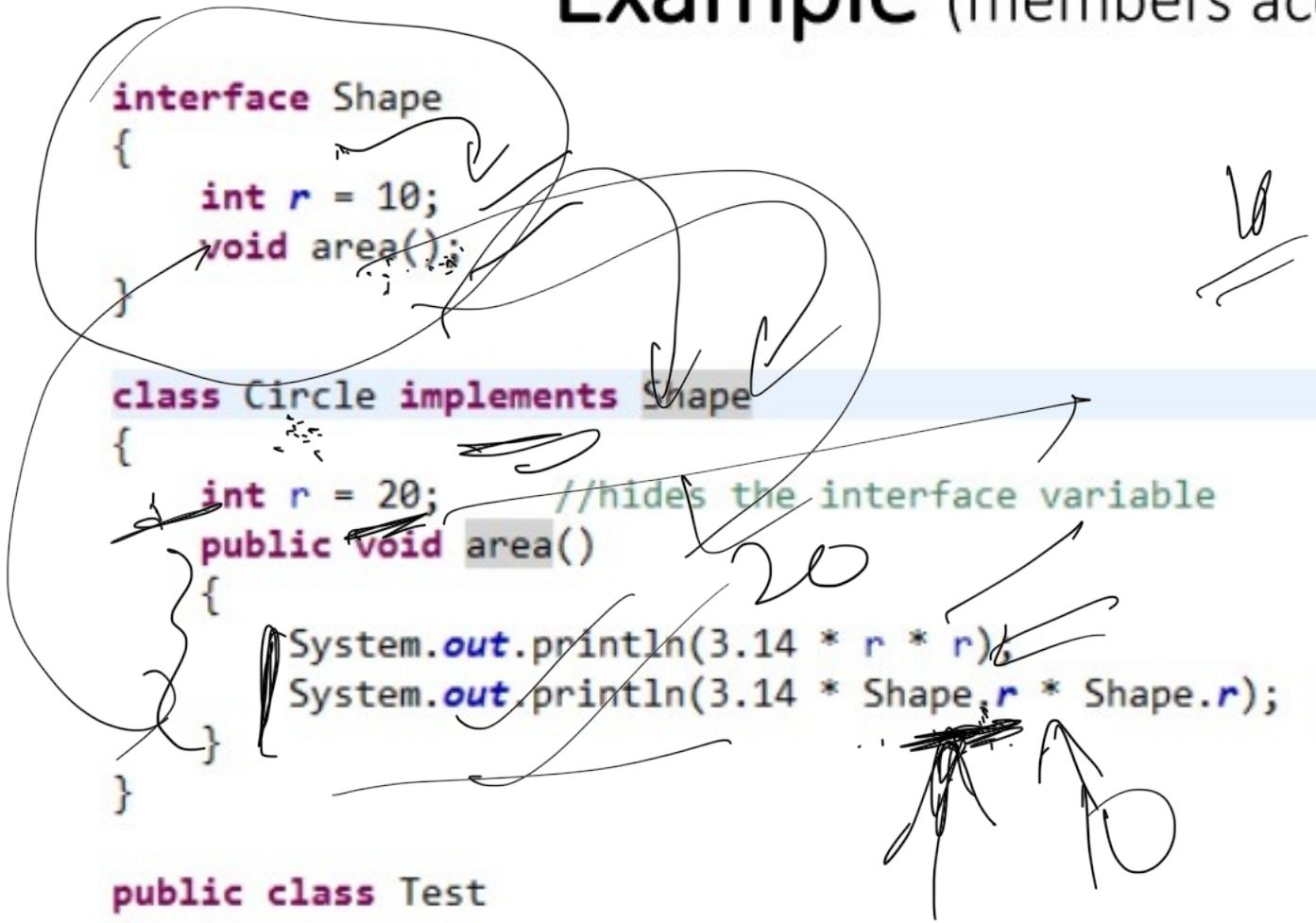


Example (members accessibility)

```
interface Shape
{
    int r = 10;
    void area();
}

class Circle implements Shape
{
    int r = 20;      //hides the interface variable
    public void area()
    {
        System.out.println(3.14 * r * r);            //accesses instance variable "r"
        System.out.println(3.14 * Shape.r * Shape.r);  //accesses interface variable "r"
    }
}

public class Test
{
    public static void main(String args[])
    {
        Circle c = new Circle();
        c.area();
    }
}
```



Example (Bank)

```
interface Bank{
float rateOfInterest();
}
class SBI implements Bank{
public float rateOfInterest(){return 9.15f;}
}
class PNB implements Bank{
public float rateOfInterest(){return 9.7f;}
}
class TestInterface2{
public static void main(String[] args){
Bank b=new SBI();
System.out.println("ROI: "+b.rateOfInterest());
}}
```

Example (Shape)

```
interface Shape
{
    public double getArea();
}

class Rectangle implements Shape
{
    double length, width;
    Rectangle(double length, double width)
    {
        this.length = length;     this.width = width;
    }

    public double getArea()       //implementing function of interface
    {   return (length * width);   }
}

class Circle implements Shape
{
    double radius;
    Circle(double radius)
    {
        this.radius = radius;
    }

    public double getArea()     //implementing function of interface
    {   return (3.14 * radius * radius);    }
}
```

```java
public class Test
{
    public static void main(String[] args)
    {
        Rectangle r = new Rectangle(3.1, 4.8);
        calculate(r);      //passing Rectangle object to compute area

        Circle c = new Circle(10.2);
        calculate(c);      //passing Circle object to compute area
    }

    static void calculate(Shape obj)  //function carrying interface reference variable
    {
        System.out.println("Area: " + obj.getArea());
    }
}
```

interface vs. **class**

An interface is different from a class in several ways, including:

- We cannot instantiate an interface.
- An interface does not contain any constructors.
- All of the methods in an interface are abstract.
- An interface is not extended by a class; it is implemented by a class.
- An interface can extend multiple interfaces.

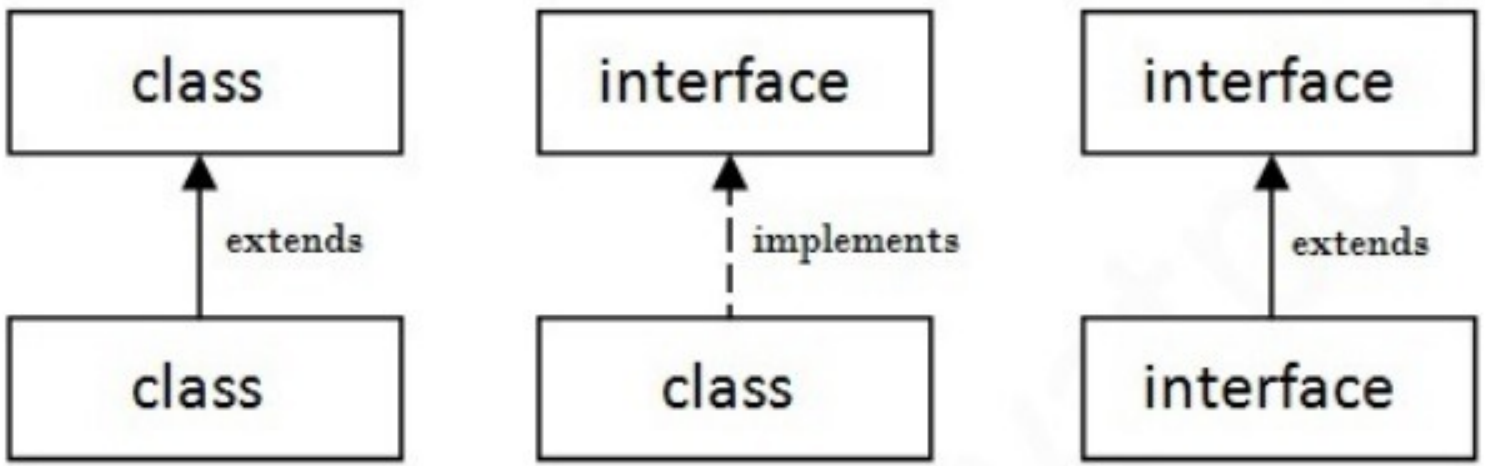
interface vs. **class**

An interface is different from a class in several ways, including:

- We cannot instantiate an interface.
- An interface does not contain any constructors.
- All of the methods in an interface are abstract.
- An interface is not extended by a class; it is implemented by a class.
- An interface can extend multiple interfaces.

Understanding relationship between classes and interfaces

As shown in the figure given below, a class extends another class, an interface extends another interface but a **class implements an interface**.



Multiple inheritance in Java by interface

If a class implements multiple interfaces, or an interface extends multiple interfaces i.e. known as multiple inheritance.

Multiple Inheritance in Java

Example (multiple inheritance using interfaces)

```
interface Printable{
void print();
}
interface Showable{
void show();
}
class A7 implements Printable,Showable{
public void print(){System.out.println("Hello");}
public void show(){System.out.println("Welcome");}

public static void main(String args[]){
A7 obj = new A7();
obj.print();
obj.show();
 }
}
```

Example (inheritance in interfaces)

A class implements interface but one interface extends another interface .

```
interface Printable{
void print();
}
interface Showable extends Printable{
void show();
}
class TestInterface4 implements Showable{
public void print(){System.out.println("Hello");}
public void show(){System.out.println("Welcome");}

public static void main(String args[]){
TestInterface4 obj = new TestInterface4();
obj.print();
obj.show();
 }
}
```

Difference between abstract class and interface

Abstract class and interface both are used to achieve abstraction where we can declare the abstract methods. Abstract class and interface both can't be instantiated. But there are many differences between abstract class and interface that are given below.

Abstract class	Interface
1) Abstract class can **have abstract and non-abstract** methods.	Interface can have **only abstract** methods. Since Java 8, it can have **default and static methods** also.
2) Abstract class **doesn't support multiple inheritance**.	Interface **supports multiple inheritance**.
3) Abstract class **can have final, non-final, static and non-static variables**.	Interface has **only static and final variables**.
4) Abstract class **can provide the implementation of interface**.	Interface **can't provide the implementation of abstract class**.
5) The **abstract keyword** is used to declare abstract class.	The **interface keyword** is used to declare interface.

6) **Example:**
public abstract class Shape{
public abstract void draw();
}
	Example:
	public interface Drawable{
	void draw();
	}

Default method in interface

Default methods

- Default methods are defined inside the interface and tagged with **default** are known as default methods.
- These methods are non-abstract methods.
- Since Java 8, we can have method body in interface. But we need to make it **default**.

Example

```
interface Shape
{
    static void color(){
        System.out.println("Red");
    }
    double area();
}

class Circle implements Shape
{
    double radius;
    public double area(){
        return 3.14*radius*radius;
    }
}

public class Test
{
    public static void main(String args[]){
        Circle c = new Circle();
        System.out.println(c.area());
        //c.color();  //error, since static method is exclusive to Shape
        Shape.color();
    }
}
```

Example (multiple inheritance using interfaces with default methods)

```
interface Shape1
{
    default void area()
    {
        System.out.println("Shape-1 method");
    }
}

interface Shape2
{
    default void area()
    {
        System.out.println("Shape-2 method");
    }
}

class Circle implements Shape1, Shape2
{
    //error, cannot use duplicate default methods from multiple interfaces
}
```

In order to remove ambiguity, override the method in the class as:

```
public void area()
{
    Shape1.super.area();     //to call method of Shape1
}
```

static method in interface

static method

- Since Java 8, you can define static methods in interfaces.
- Java interface static method is part of interface, we can’t use/access it using class objects.
- Java interface static method helps us in providing security by not allowing implementation classes to override them.

Example

```
interface Shape
{
    static void color(){
        System.out.println("Red");
    }
    double area();
}

class Circle implements Shape
{
    double radius;
    public double area(){
        return 3.14*radius*radius;
    }
}

public class Test
{
    public static void main(String args[]){
        Circle c = new Circle();
        System.out.println(c.area());
        //c.color();  //error, since static method is exclusive to Shape
        Shape.color();
    }
}
```

Example (multiple inheritance using interfaces with static methods)

```java
interface Shape1
{
    static void area()
    {
        System.out.println("Shape-1 method");
    }
}

interface Shape2
{
    static void area()
    {
        System.out.println("Shape-2 method");
    }
}
```

```java
class Circle implements Shape1, Shape2
{
    void fun()
    {
        //area();      //error, method not defined for Circle class
        Shape1.area();
        Shape2.area();
    }
}

public class Test
{
    public static void main(String args[])
    {
        Circle c = new Circle();
        c.fun();
    }
}
```